



UNIVERSITÀ
DI TORINO

Relazione di Progetto: Simulazione Mensa "Oasi del Golfo"

Corso di Laboratorio di Sistemi Operativi

Componenti del Gruppo:

1) Alessandro Aprile

- Matricola: 1133277
- Email Istituzionale: alessandro.aprile435@edu.unito.it

2) Marco Cretu

- Matricola: 1103347
- Email Istituzionale: marco.cretu@edu.unito.it

1. Introduzione e Obiettivi del Progetto

Il progetto presentato implementa la simulazione del servizio di una mensa studentesca e aziendale denominata "Oasi del Golfo", sviluppata in linguaggio C in ambiente UNIX.

L'obiettivo principale è la modellazione di un sistema concorrente e multiprocesso, capace di gestire l'interazione sincronizzata tra utenti, operatori delle stazioni di distribuzione e il cassiere.

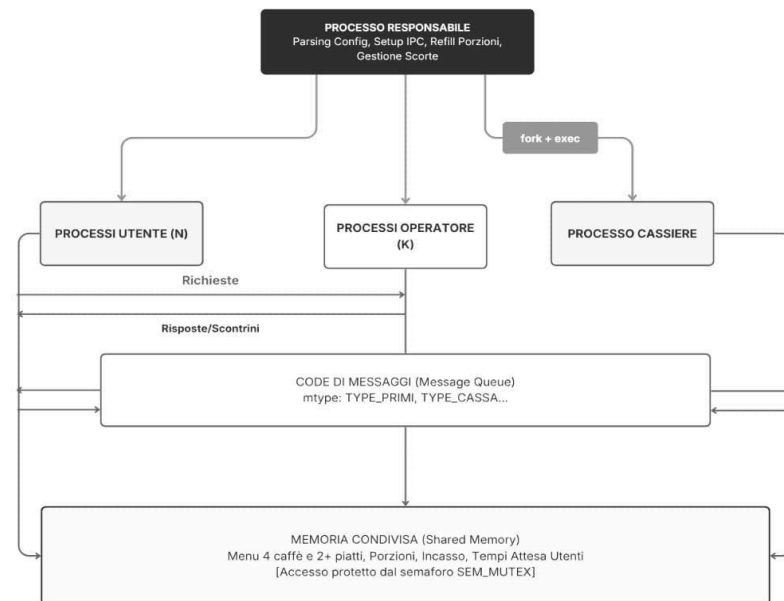
Il sistema soddisfa tutti i requisiti della "versione minima" descritta nelle specifiche del progetto, gestendo correttamente il ciclo di vita della mensa, le pause del personale, il rimpiazzo delle porzioni (refill), i tempi di erogazione stocastici e l'eventuale chiusura forzata per *overload* (sovraccarico di utenti). Il codice è stato sviluppato in moduli separati (src, inc, obj, bin) ed è interamente compilato tramite l'ausilio di un Makefile con direttive di controllo rigorose (-Wvla -Wextra -Werror e -D_GNU_SOURCE).

2. Architettura del Sistema e Modello dei Processi

Il sistema è governato da un processo principale (responsabile.c), che agisce da supervisore per l'intera durata della simulazione. L'architettura prevede la generazione dinamica dei seguenti processi figli tramite system call fork() ed exec():

- **Responsabile Mensa (responsabile.c):** Esegue il parsing del file di configurazione (.conf) e del menu (menu.txt), inizializza le strutture IPC (memoria condivisa, semafori, code di messaggi) e avvia i processi dello staff. Successivamente, scandisce i giorni di simulazione, innesca il timer per il *refill* e calcola le statistiche finali, monitorando le condizioni di eventuale *overload*.
- **Operatori di Stazione (operatore.c):** Processi generici che assumono un ruolo specifico (Primi, Secondi o Caffè) tramite argomenti a riga di comando. Le stazioni dei Primi e dei Secondi offrono almeno 2 opzioni di piatti, con un tempo di erogazione variabile del +/- 50% rispetto alla media; la stazione Caffè offre 4 varianti (normale, macchiato, decaffeinato, ginseng) con varianza del +/- 80% sul tempo di erogazione.
- **Cassiere (cassiere.c):** Processo dedicato alla riscossione dei pagamenti, che simula il proprio tempo di servizio e aggiorna l'incasso in mutua esclusione.
- **Utenti (utente.c):** Processi generati dal responsabile che navigano all'interno della mensa. Effettuano scelte pseudo-casuali sui pasti, comunicano con gli operatori, pagano in cassa, misurano il proprio tempo di attesa e occupano fisicamente i tavoli per la consumazione.

2.1 Architettura del Sistema e Modello dei Processi



3. Gestione della Comunicazione Inter-Processo (IPC)

Per garantire l'integrità dei dati e la corretta cooperazione tra le entità, sono stati sfruttati tre meccanismi di comunicazione System V IPC:

3.1 Memoria Condivisa (Shared Memory)

È stata definita una struttura `shared_data_t` mappata nello spazio di indirizzamento di tutti i processi. Questa memoria globale contiene:

- Lo stato di esecuzione della simulazione (`sim_running`, `sim_time`).
- Gli array del menu, comprensivi di nomi, prezzi e porzioni rimanenti. Come richiesto, sono implementate almeno 2 scelte per i piatti e le 4 tipologie obbligatorie per il caffè.
- I contatori statistici: totale incassato, utenti serviti/rinunciati, piatti erogati e somma dei tempi di attesa.

3.2 Code di Messaggi (Message Queues)

Per evitare attese attive (busy-waiting) e modellare fedelmente le code fisiche davanti alle stazioni, si è optato per le Message Queues.

- **Invio:** Un utente invia una struttura `msg_t` impostando il campo `mtype` in base alla stazione di destinazione (es. `TYPE_SECONDI`).
- **Ricezione:** L'operatore si pone in ascolto bloccante solo sui messaggi diretti alla sua postazione.
- **Risposta:** L'operatore, elaborata la richiesta, invia un messaggio di ritorno impostando il campo `mtype` con il PID del processo utente (`sender_pid`), garantendo che ogni cliente riceva esattamente la risposta al proprio ordine (esito 1 se servito, 0 se porzioni esaurite).

3.3 Gestione a Semafori (Semaphores)

È stato allocato un set vettoriale di 7 semafori per modellare sia la mutua esclusione sia le risorse fisiche (spazio in mensa):

1. **SEM_MUTEX:** Semaforo binario per proteggere gli aggiornamenti dei contatori nella Memoria Condivisa.
2. **SEM_TAVOLI:** Semaforo contatore inizializzato al numero massimo di posti a sedere (NOF_TABLE_SEATS). Gli utenti effettuano una `semop(-1)` prima di mangiare.
3. **SEM_PRIMI, SEM_SECONDI, SEM_COFFEE, SEM_CASSA:** Semafori che rappresentano lo spazio fisico dietro al bancone (NOF_WK_SEATS). Ogni operatore/cassiere deve acquisire una postazione prima di iniziare il turno.
4. **SEM_SYNC:** Utilizzato come barriera di sincronizzazione (*rendez-vous*). Assicura che l'afflusso degli utenti inizi solo quando l'intero staff ha occupato la propria postazione ed è pronto a ricevere ordini.

4. Scelte Progettuali e Gestione delle Criticità

Durante lo sviluppo, particolare attenzione è stata posta alla gestione delle eccezioni e alla stabilità del sistema operativo sottostante.

4.1 Prevenzione dei Deadlock

Tutte le operazioni semaforiche relative a risorse ad accesso prolungato (es. l'occupazione dei tavoli da parte degli utenti e delle stazioni da parte degli operatori) sono state implementate utilizzando la flag `SEM_UNDO`. In questo modo, qualora la simulazione venga interrotta bruscamente dal Responsabile (ad esempio per il raggiungimento del limite di tolleranza di *Overload* in coda), il Kernel rilascerà automaticamente le risorse trattenute dai processi terminati, evitando stalli permanenti nel sistema.

4.2 Proporzionalità Temporale e Rand

Per non bloccare eccessivamente l'esecuzione sui processori reali e rispettare il tempo simulato, il codice utilizza la variabile di configurazione scalare `N_NANO_SECS`. Le pause degli operatori, i tempi di consumazione ai tavoli e il tempo del cassiere vengono generati tramite la funzione `usleep()`, calcolata dinamicamente come sottomultiplo di `N_NANO_SECS`. La variabilità richiesta per il confezionamento dei piatti (es. +/- 50% o +/- 80%) è calcolata tramite la funzione `rand()`, inizializzata in ogni processo con un *seed* unico generato tramite lo XOR tra il PID del processo e l'orologio di sistema (`srand(getpid() ^ time(NULL))`).

4.3 Registrazione Dinamica dei Tempi di Attesa

Per calcolare la media del tempo speso dagli utenti, ogni processo utente interroga l'orologio ad alta risoluzione del sistema POSIX tramite `clock_gettime(CLOCK_MONOTONIC)` un istante prima di inserire il messaggio in coda, e immediatamente dopo aver ricevuto la risposta dall'operatore. Il delta misurato in microsecondi viene cumulato nella memoria condivisa, permettendo al responsabile di stampare a fine giornata statistiche accurate.

4.4 Robustezza sulla Disconnessione

Un bug logico intrinseco affrontato riguarda i cosiddetti "Utenti Fantasma": processi che rimangono in coda durante la transizione tra due giornate di simulazione. Per evitare che l'azzeramento della coda di fine giornata sfasi i contatori (portandoli a valori negativi quando gli utenti serviti tentano di rimuovere sé stessi dalla coda il giorno successivo), è stata implementata una guardia condizionale sotto Mutex (`if users_waiting > 0`) prima di permettere a un utente di sottrarsi al conteggio generale. Inoltre, in fase di chiusura simulazione, il responsabile implementa una funzione `cleanup()` che intercetta i segnali SIGINT e SIGTERM rimuovendo diligentemente tutta la memoria IPC allocata tramite `shmctl`, `semctl` e `msgctl`.